We can also pass several additional parameters in the `match` query's full form. So far, we have discussed a shortened form of the `match` query. Here is an example of the full form:

```
GET books/_search
{
  "query": {
    "match": {
      "FIELD": {
        "query":"<SEARCH TEXT>",
        "<parameter>":"<MY_PARAM>",
      }
    }
  }
}
```

Declares FIELD as an object with additional parameters

The query attribute holds the search text.

The parameter (analyzer, operator, prefix_length, fuzziness, etc.) expects a value to be set.

We can search across multiple indexes by providing comma-separated indexes in the search URL:

```
GET new_books,classics,top_sellers, crime* /_search
{
  ...
}
```

Any number of indexes can be provided when invoking the `_search` endpoint, including wildcards.

> **NOTE** If we omit the index (or indexes) in the search request, we effectively search the entire index. For example, `GET _search { ... }` searches across all the indexes in the cluster.

10.5.2 Searching using a match query

Now that we know the format for a `match` query, let's look at an example where we want to search for Java books with *Java* in the `title` field. The following listing sets the `title` field to the word "Java" as the text to search.

Listing 10.4 Searching for Java books

```
GET books/_search
{
  "query": {
    "match": {
      "title": "Java"
    }
  }
}
```

The match query in action

Sets the search criteria, searching for the word "Java" in the title field

We create a `match` query with the criterion of searching for a word in a `title` field. Elasticsearch fetches all documents that match the word *Java* in the `title` field, as expected.